International University of Grand Bassam



Compagnie Ivoirienne D'Electricité



Capstone Final Project Report

Object Tracking and Re-identification

Author: Ibo Schilo Zokou (ID: 105596)

A thesis submitted in fulfillment of the requirements for the joint IUGB Bachelor of Science degree in Data Science

March 31, 2025

"I am the master of my fate, I am the captain of my soul"
from Invictus*, by William Ernest Henley*

Dedication

Before proceeding, I would like to express my heartfelt gratitude to Jesus for everything He has done for me throughout my journey at IUGB. I am deeply thankful for the strength, knowledge, health, wisdom, and power that He has bestowed upon me. His guidance has allowed me to overcome all obstacles and to complete this journey successfully. I would like to take this opportunity to extend my sincere gratitude to my entire family, the IBO family, with special thanks to my father, Ibo Aime ,and my mother, Ibo Patricia, who have been a constant source of inspiration. Their unwavering support, invaluable advice, daily financial assistance, and prayers have been essential to my journey. I also wish to express my heartfelt appreciation to my brothers and sister. A warm and profound thanks to the International University of Grand-Bassam (IUGB), including all instructors and staff. I am particularly grateful to my esteemed Capstone Instructor, Dr. KALA Jules Raymond, for his guidance and advice throughout this work. Finally, I extend my deepest gratitude to everyone who has supported, helped, and guided me along the way. May GOD KEEPS BLESSING YOU!!!

Contents

1 Introduction **5**

2 Introduction to Computer Vision **7**

2.1 Definition 7
2.2 History/Origin 7
2.3 Goal 8
2.4 Importance of Computer Vision 8
2.5 Real-World Applications of Computer Vision 9
2.6 Process of Computer Vision 10
2.7 Tools and Frameworks in Computer Vision 11
2.8 Challenges and Limitations in Computer Vision 13
2.9 State of the Art in Computer Vision 14

3 Deep Learning **17**

3.1 Definition 17
3.2 History/Origin 18
3.3 Goal 18
3.4 Importance of Deep Learning 19
3.5 Real-World Applications of Deep Learning 19
3.6 Process of Deep Learning 20
3.7 Deep Learning Tools 22
3.8 Challenges and Limitations of Deep Learning 23
3.9 State-of-the-Art (SOTA) in Deep Learning 24
3.10 Computer Vision vs Deep Learning 26

4 Object Tracking and Re identification **29**

4.1 Introduction 29

4.2 Theoretical Background 29

4.3 Methodology for Object Tracking and Re-Identification 32

4.3.1 Data Collection 32

4.3.2 Object Detection-Based Tracking Implementation 33

4.3.3 TorchReID: Exploring the TorchReID Model Zoo 35

4.3.4 Dimensionality Reduction with Principal Component Analysis (PCA) 37

4.3.5 K-Nearest Neighbor (KNN) for Classification 37

4.3.6 Anomaly Detection with Autoencoders 37

4.4 Result and Discussion 39

4.4.1 Feature Extraction for Re-Identification 39

4.4.2 Challenges in Identifying New Individuals 42

4.4.3 Anomaly Detection Approach for Identifying New Individuals 43

4.4.4 Person Re-Identification Using K-Nearest Neighbors (KNN) 45

4.4.5 Discussion of Results and Limitations 45

Bibliography **49**

Chapter 1

Introduction

Object tracking and re-identification (Reid) have become fundamental tasks in computer vision, essential for applications ranging from video surveillance to autonomous systems. Object tracking involves the continuous detection and localization of an object within a sequence of frames, while re-identification focuses on recognizing the same object across multiple non-overlapping cameras or after significant temporal gaps. Together, these tasks enable the development of advanced multi-object, multi-camera tracking systems that are critical for maintaining situational awareness in complex environments such as airports, shopping malls, and smart cities. The roots of object tracking can be traced back to the early works of [8], who proposed an optical flow-based method for tracking features across frames. Over the decades, numerous approaches have been developed, ranging from traditional methods like Kalman Filters and Mean-Shift to more recent deep learning-based techniques such as YOLO which is a popular real-time object detection system that can detect multiple objects in an image or video with high speed and accuracy. The advent of convolutional neural networks (CNNs) revolutionized the field, enabling more accurate tracking by learning robust feature representations from vast amounts of data. The introduction of [1] marked a significant milestone, allowing for the efficient and effective tracking of objects by comparing template and candidate regions. Re-identification, while sharing similarities with object tracking, presents its own set of challenges, particularly due to variations in appearance, pose, and lighting across different camera views. Early ReID methods relied heavily on handcrafted features, such as color histograms and texture descriptors, which, despite their simplicity, struggled to cope with the complexity of real-world scenarios. The introduction of deep learning into ReID, particularly with the work of [28], demonstrated the potential of CNNs to learn discriminative features that are invariant to such variations. Their work introduced a deep ranking model that effectively addressed

the issue of inter-class similarity, significantly improving re-identification performance. In recent years, the integration of deep learning with both object tracking and re-identification has led to the development of more sophisticated systems capable of handling the intricacies of real-world environments. Our project is mainly based on the work of [30] with the library torchreid where we took anything that we want in order to do a re-identification process. Ultimately, this work seeks to advance the current understanding of object tracking and re-identification, offering new insights and solutions that could enhance the performance of intelligent surveillance and monitoring systems.The problematic is now how can we efficiently perform object tracking and re-identification in real time ? By solving this problem we aim to contribute to the ongoing research efforts in computer vision and deepen the integration of these technologies into practical, real-world applications.

Chapter 2

Introduction to Computer Vision

2.1 Definition

Computer vision is a branch of artificial intelligence (AI) that empowers computers and systems to extract and interpret meaningful information from digital images, videos, and other visual inputs [5]. By leveraging machine learning techniques, particularly neural networks, computer vision enables systems to analyze visual data, make informed decisions, and provide recommendations based on detected patterns or issues. This technology effectively gives computers the capability to "see" and understand the world in a manner similar to human vision.

2.2 History/Origin

According to [6] during the 1960s , researchers began creating algorithms to process and analyze visual data, though progress was constrained by the limited computational power of the time. By the 1970s, more advanced algorithms for image processing and pattern recognition were developed. A significant advancement during this period was the introduction of the Hough transform, which facilitated the detection of lines and other geometric shapes in images. In the 1980s and 1990s, the focus shifted to developing machine learning algorithms for computer vision, enabling computers to learn from data and gradually improve their accuracy. One of the most notable achievements from this era was the Viola-Jones face detection algorithm, introduced in 2001. In the 2000s and 2010s, deep learning algorithms brought about a revolution in computer vision by allowing computers to learn hierarchical representations of visual data. The advent of convolutional neural networks (CNNs) and other deep learning methods enabled computers to achieve unprecedented

accuracy in tasks such as object recognition, movement tracking, and other complex visual analyses. Today, computer vision is expanding rapidly, with applications spanning autonomous vehicles, medical imaging, robotics, and more. The development of efficient algorithms, such as YOLO (You Only Look Once) and Fast R-CNN, has enabled real-time object detection and tracking, expanding the use cases for computer vision in areas like autonomous vehicles and augmented reality. As artificial intelligence continues to advance, computer vision is poised to grow further, transforming how we interact with visual information.

2.3 Goal

The aim of computer vision is to enable computers to analyze and comprehend visual data in a way that mirrors how human eyes and brains perceive and interpret such information, and to utilize this understanding to make intelligent decisions based on that data.

2.4 Importance of Computer Vision

Computer Vision is important because it permits understanding and analyzing visual information, unlocking numerous practical applications across various industries. Here are key reasons why it holds significance:

- **Automation of Visual Tasks:** Computer vision automates tasks that require visual interpretation, such as quality inspection, object sorting, and facial recognition, reducing human labor and increasing efficiency in fields like manufacturing, retail, and security.
- **Scalability:** With computer vision, tasks that involve analyzing vast amounts of visual data—such as surveillance, satellite imagery analysis, and crowd monitoring—can be scaled efficiently, far beyond what manual human efforts can achieve.
- **Improved Accuracy and Speed:** Computer Vision can process large volumes of data with better accuracy and speed than humans. For example, in medical imaging, it aids in early diagnosis and enhances the accuracy of detecting abnormalities like tumors.

2.5 Real-World Applications of Computer Vision

Computer Vision has numerous real-world applications across various industries, significantly transforming business operations and industry practices. Here are some examples:

- **Autonomous Vehicles:** Computer vision is critical in enabling self-driving cars to understand their surroundings by detecting pedestrians, vehicles, road signs, and obstacles [5]. Companies like Tesla, Waymo, and Uber are using computer vision for real-time navigation.



Figure 2.1: Sample Image with Semantic Segmentation for Road Scene from [17]

- **Healthcare:** With computer vision, medical imaging diagnostics are possible. Algorithms analyze X-rays, MRIs, and CT scans to detect diseases such as cancer, tumors, and fractures. For example, AI-based diagnostic tools assist radiologists in identifying abnormalities faster and with greater accuracy.
- **Augmented Reality and Virtual Reality:** Computer vision powers AR apps like Snapchat and IKEA's AR app, allowing users to visualize how furniture will look in their homes by overlaying objects onto the real world. It is also helpful in virtual gaming, where computer vision tracks user movements and gestures, enhancing the immersive experience in gaming simulations.
- **Smart Cities:** Computer vision is used in smart traffic systems to monitor road conditions, manage traffic flow, detect accidents, and issue warnings or reroute vehicles during congestion. Vision systems in smart cities help law enforcement monitor

large crowds during events, detect potential threats, and improve emergency response times.

2.6 Process of Computer Vision

The process of computer vision involves a series of steps to enable a computer to analyze, interpret, and make decisions based on visual data, such as images or videos. Below is an outline of the typical computer vision process:

- **Image Acquisition:** The first step is capturing the visual data, typically through cameras, sensors, or other devices. This data could be in the form of digital images. The raw visual data may be preprocessed to improve its quality, which may involve adjusting brightness, contrast, and resolution, or removing noise from the image.
- **Image Processing:** Image processing techniques such as noise reduction and filtering are used to remove unwanted noise that can interfere with analysis. Another technique is image resizing, where images are adjusted to a standard dimension suitable for further analysis, especially when using machine learning models. Color space conversion may also be applied, depending on the task requirements.
- **Feature Extraction:** Edge detection algorithms like Sobel or Canny are used to identify and highlight the boundaries of objects within the images. Shape and texture detection algorithms, such as the Hough Transform, can be applied to detect geometric shapes in the image. Texture-based features like gradients, corners, or patterns are also extracted.
- **Object Detection and Recognition:** This step includes object localization, where the algorithm identifies and locates objects in the image by drawing bounding boxes around them. The detected objects are then classified into specific categories. For example, a system can recognize whether an object is a car, a pedestrian, or an animal.
- **Pattern Recognition:** The system is designed to recognize specific patterns within an image, such as faces, vehicles, or other objects. Algorithms, including Convolutional Neural Networks (CNNs), are widely used for this task, especially in applications like facial recognition, object detection, and anomaly detection.

- **Post Processing:** After data processing and analysis, informed decisions must be made based on the insights gained. For instance, in an autonomous vehicle, the system might determine the need to stop or change lanes in response to detected objects, such as pedestrians or other vehicles. The result of the decision-making process is used to trigger an action or provide feedback to users or other systems, often in real-time, as in robotics or autonomous systems.

- **Deployment:** Once the computer vision system is developed and tested, it can be deployed in real-world environments. It is often integrated into applications such as autonomous driving systems, medical imaging tools, or industrial robotics.

2.7 Tools and Frameworks in Computer Vision

The field of computer vision relies on various tools, libraries, frameworks, and hardware that facilitate tasks such as image processing, object detection, segmentation, and machine learning. Below is an overview of the most important tools and frameworks used in computer vision:

- **Programming Languages:** The most popular programming language is Python because it's easy to use and has a vast ecosystem of libraries for image processing and machine learning. C++ is also used for performance-critical tasks, especially in real-time computer vision applications like robotics or embedded systems.

- **Libraries and Frameworks:** Popular image processing libraries include OpenCV (Open Computer Vision Library), which offers a wide range of functions such as image transformation, filtering, edge detection, face detection, and feature extraction. Pillow is another Python library used for opening, manipulating, and saving image files. Scikit-image, part of the broader scikit-learn ecosystem, focuses on image processing algorithms and includes functions for segmentation, color manipulation, and image enhancement.

- **Deep Learning Frameworks:** Useful frameworks for deep learning include TensorFlow, which is robust for computer vision tasks with high-level APIs like tf.Keras and TensorFlowHub for pre-trained models. PyTorch is also widely used in research and production for computer vision due to its flexibility and dynamic computational graph.

- **Object Detection Frameworks:** YOLO (You Only Look Once) with models like YOLOv9 is popular for applications in autonomous vehicles, surveillance, and robotics. Detectron2 is another widely used framework for tasks like object detection, instance segmentation, and keypoint detection.
- **Segmentation and Scene Understanding Tools:** DeepLab is a state-of-the-art semantic segmentation model that performs pixel-level labeling of images, available in TensorFlow and PyTorch implementations. Mask R-CNN, used for instance segmentation, extends Faster R-CNN and is available through TensorFlow, PyTorch, and Detectron2.
- **3D Vision and Point Cloud Processing:** Open3D is a library for 3D data processing, including point clouds, meshes, and RGB-D images, supporting tasks like 3D reconstruction, SLAM, and surface alignment. PCL (Point Cloud Library) is used for processing 3D point cloud data, providing tools for filtering, segmentation, and object recognition in 3D data.
- **Annotation and Labeling Tools:** LabelImg is commonly used for object detection tasks, and other annotation tools include CVAT, Labelbox, and Supervisely.
- **Pre-trained Model Hubs:** TensorFlow Hub is a repository of pre-trained machine learning models for TensorFlow, including various computer vision models. PyTorch Hub serves a similar purpose for PyTorch models.
- **Data Augmentation and Synthetic Data Generation:** Albumentations is a popular image augmentation library for machine learning, widely used in computer vision tasks to increase dataset diversity by applying transformations such as cropping, flipping, rotation, and color adjustments. Augmentor is another Python package for augmenting image datasets with transformations like rotation, flipping, and random noise to improve generalization.
- **Edge and Embedded Systems for Computer Vision:** Nvidia Jetson is Nvidia's platform for deploying computer vision models on the edge. It supports real-time object detection, classification, and other computer vision tasks in resource-constrained environments. Google Coral is another edge AI platform, including the Coral USB Accelerator, which runs TensorFlow Lite models for vision tasks on edge devices.

- **Visualization and Model Evaluation Tools:** TensorBoard allows users to monitor metrics like loss and accuracy during model training and also visualizes images, filters, and activations for computer vision tasks. Matplotlib, a Python library, is essential for visualizing images, bounding boxes, and segmentation masks, aiding in the analysis of computer vision model outputs.

- **Datasets for Computer Vision:** COCO (Common Objects in Context) is a large dataset for object detection, segmentation, and captioning, containing over 200,000 labeled images across 80 object categories. ImageNet is another massive dataset used for image classification, with state-of-the-art models like ResNet and EfficientNet pre-trained on it.

2.8 Challenges and Limitations in Computer Vision

Computer Vision faces numerous limitations and challenges, both in technical development and real-world applications:

- **Data Quality and Availability:** High-quality and diverse datasets are crucial for training accurate models, but collecting such data is challenging, particularly for niche applications. Additionally, biases in datasets can affect model performance and fairness.

- **Computational Complexity:** Processing visual data requires immense computational power. This complexity increases with tasks like real-time video processing, 3D image analysis, or high-resolution image segmentation. Scaling these processes often requires advanced hardware like GPUs or Edge AI devices, which can be costly and resource-intensive.

- **Generalization Across Environments:** Many computer vision models struggle with generalization. A model trained in a controlled setting may perform poorly in dynamic, real-world environments due to variations in lighting, angles, occlusion, and noise.

- **Ethical Concerns:** Privacy issues and ethical concerns arise when deploying computer vision systems, especially in surveillance or facial recognition technologies. There are debates surrounding the misuse of these systems, particularly in contexts where individuals' consent or privacy rights are at risk.

- **Data Privacy and Security:** The need to process sensitive or personal data (such as medical images or facial recognition) raises concerns about ensuring data privacy, secure transmission, and storage. Regulations like GDPR in Europe place constraints on how such data can be used.

2.9 State of the Art in Computer Vision

The current state of the art in computer vision is rapidly advancing, with breakthroughs across various domains such as object detection, image classification, and generative AI. Some of the most notable trends and technologies shaping the field in 2024 include:

- **YOLOv8 and Object Detection:** YOLOv8 [23] represents the pinnacle of progress in the realm of computer vision, standing as the new state-of-the-art in object detection models. Developed by Ultralytics, this version of the YOLO model series brings forth significant advancements over its predecessor, YOLOv5, and earlier YOLO versions. YOLOv8 comes equipped with a range of new features that enhance its detection capabilities, making it more accurate and efficient than ever before.

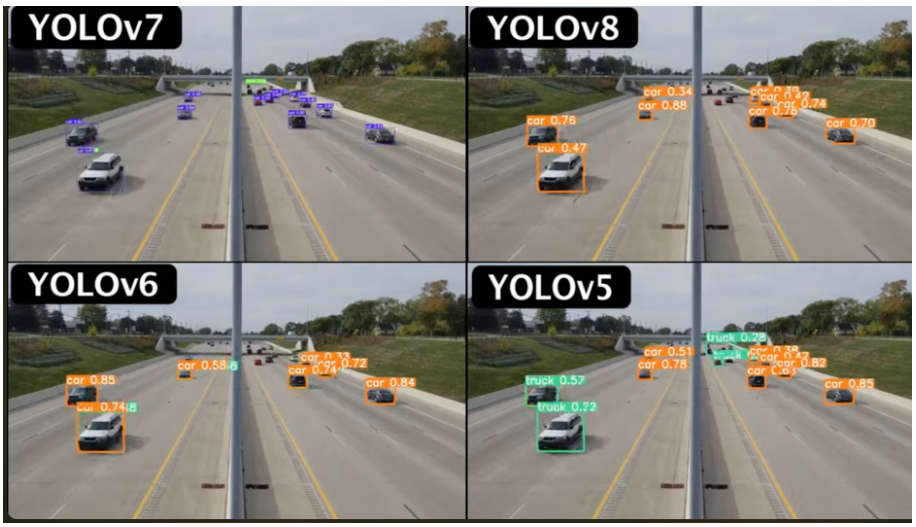


Figure 2.2: Comparison of YOLO Object Detection Models (YOLOv5 to YOLOv8) in Real-Time Vehicle Detection on Highway from [15]

- **Deep Learning for Image Classification:** Convolutional Neural Networks (CNNs) continue to dominate image classification. With models like AlexNet, VGGNet, and ResNet, CNNs have brought high accuracy to tasks like face recognition and medical

image analysis. In 2024, advanced architectures such as transformers are gaining ground for even better performance.

- **Generative AI in Computer Vision:** The integration of generative AI techniques in computer vision focuses on applications such as image stylization, data enhancement, and visual mapping. Generative models like GANs present new methods and challenges in visualization and image processing.

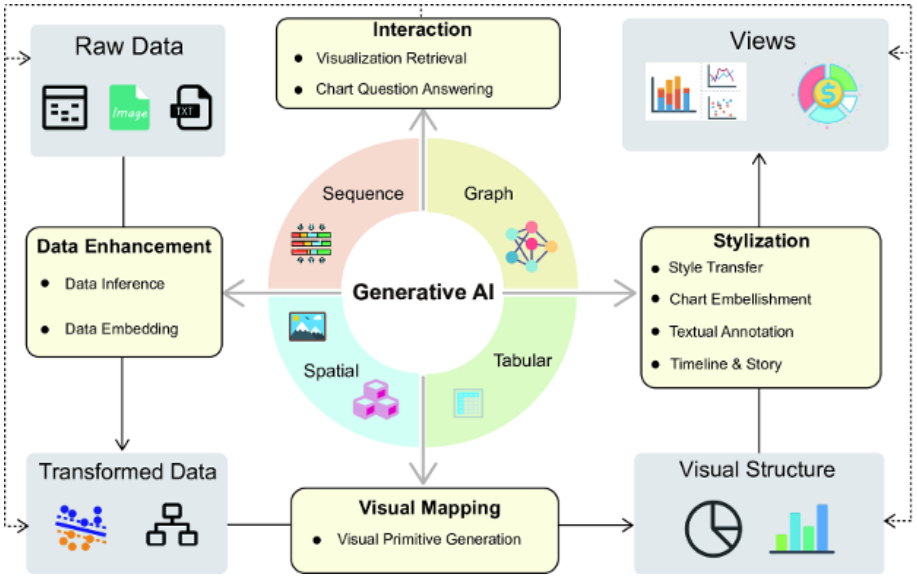


Figure 2.3: Generative AI Workflow for Data Visualization and Enhancement from [24]

- **Augmented Reality (AR) and Robotics:** The integration of computer vision with AR is becoming more prevalent, particularly in consumer applications. Moreover, robotics is increasingly leveraging vision-language models to enhance AI agents' responsiveness and intuitiveness in both industrial and daily tasks.

- **Sophisticated Satellite Vision:** Advances in satellite imagery, fueled by computer vision, are enabling more detailed monitoring of terrestrial phenomena, such as deforestation, urban sprawl, and marine environments. The enhanced resolution provided by these technologies is crucial for environmental monitoring and management.

Summary

We saw a comprehensive overview of computer vision, beginning with its definition and exploring its historical development. It outlines the field's primary goals, emphasizing

its importance in automating visual interpretation and enhancing various technological applications.

We also discussed the broad usage of computer vision across industries and how computer vision operates, detailing the key steps involved in interpreting visual data. Additionally, we examined the tools and techniques utilized, from traditional image processing to advanced deep learning models like convolutional neural networks (CNNs) and transformers.

In the next chapter, we will explore more about deep learning.

Chapter 3

Deep Learning

3.1 Definition

[14] Deep learning (DL) is a high-dimensional data reduction technique used for constructing complex input-output models. It is a form of machine learning that employs hierarchical layers of latent features to process data. Unlike traditional inferential methods, deep learning is predictive and often functions as a "black-box" methodology, meaning it can handle high-dimensional function estimation without requiring explicit programming for each task. DL is widely applied in areas like artificial intelligence, image processing, robotics, and automation.

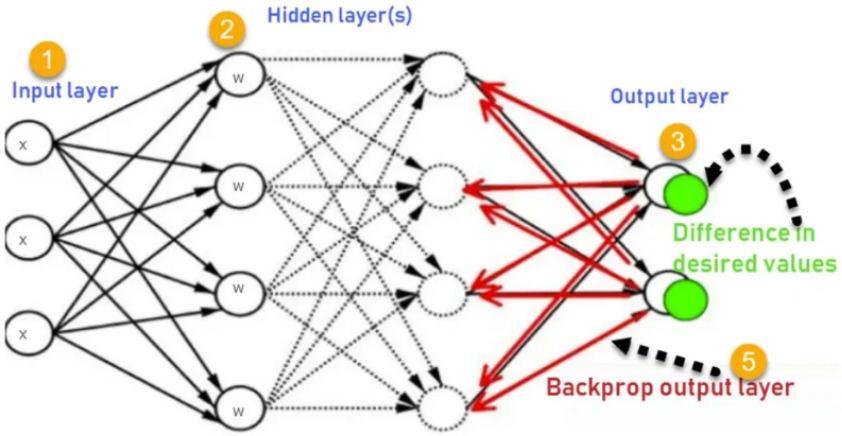


Figure 3.1: Structure and Backpropagation Process in a Neural Networkt from [9]

3.2 History/Origin

Deep learning's journey [21] began in the mid-20th century with the advent of artificial neural networks, inspired by the human brain's structure. Early efforts, like the perceptron in the 1950s, showed promise but were limited by computational power and data availability. The field remained relatively dormant until the 1980s when backpropagation was introduced, enabling deeper networks to be trained more effectively. However, it wasn't until the 2010s that deep learning truly flourished, driven by the convergence of big data, powerful GPUs, and improved algorithms. The 2010s marked the beginning of the deep learning revolution. In 2012, Geoffrey Hinton and his team, including Alex Krizhevsky and Ilya Sutskever, introduced AlexNet, a deep CNN that won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) by a significant margin. AlexNet's success demonstrated the power of deep learning for image classification and ignited widespread interest in the field. Following AlexNet, deep learning architectures such as VGGNet, GoogLeNet (Inception), ResNet, and DenseNet further pushed the boundaries of computer vision, achieving state-of-the-art performance on various benchmarks. These architectures introduced innovations such as deeper networks, residual connections, and more efficient use of computational resources. In addition to CNNs, advancements in natural language processing (NLP) were driven by deep learning models like word2vec, GloVe, and later, transformers. The Transformer architecture, introduced by Vaswani et al. in 2017, revolutionized NLP by enabling models to handle long-range dependencies more effectively. Transformers became the foundation for models like BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pre-trained Transformer), which achieved state-of-the-art results in a wide range of NLP tasks.

3.3 Goal

The goal of deep learning is to enable machines to automatically learn complex patterns and representations from large amounts of data [14]. By using hierarchical layers in neural networks, deep learning aims to model high-dimensional functions, making it possible for machines to perform tasks like image recognition, natural language processing, and autonomous decision-making with a level of accuracy and efficiency that rivals human capabilities.

3.4 Importance of Deep Learning

Deep learning's ability to automatically learn from vast amounts of data without manual feature engineering has made it a cornerstone of AI development, facilitating the advancement of technologies such as generative AI, computer vision, and application modernization. It is crucial for several reasons, such as:

- **Handling Complex Data:** Deep learning models, particularly neural networks, can handle vast amounts of complex, high-dimensional data, including images, text, audio, and more. They automatically learn to extract features from raw data without the need for manual feature engineering, making them particularly powerful in domains like image recognition, natural language processing (NLP), and speech recognition.
- **Scalability:** Deep learning can scale effectively with increasing data and computational power. As more data becomes available and computing resources continue to grow (e.g., GPUs, TPUs), deep learning models can leverage this to improve their performance. This scalability makes them suitable for tasks that require processing massive datasets, such as autonomous driving or large-scale recommendation systems.
- **Automation of Feature Extraction:** Traditional machine learning models often require manual feature extraction, where domain experts determine which features are relevant for the task. Deep learning models, particularly convolutional neural networks (CNNs) for images and recurrent neural networks (RNNs) or transformers for sequences, can automatically learn relevant features from the raw data, reducing the need for human intervention and expertise in feature design.
- **Handling Unstructured Data:** A significant portion of the world's data is unstructured, such as text, images, and audio. Deep learning models are particularly well-suited to extract insights from this kind of data, making them invaluable for modern data-driven applications.

3.5 Real-World Applications of Deep Learning

Deep learning has a wide range of real-world applications across multiple industries:

- **Virtual Assistants:** Virtual assistants like Siri, Alexa, and Google Assistant use deep learning to understand voice commands, provide personalized responses, and adapt to user preferences. They leverage deep learning for speech recognition, natural language processing, and even text generation to perform tasks like booking appointments, translating speech to text, and answering questions.

- **Entertainment:** Platforms like Netflix and VEVO utilize deep learning to analyze viewer preferences and deliver personalized content. For example, Netflix uses deep learning algorithms to recommend shows and movies based on a user's viewing history. Deep learning is also used in video analysis for sports highlights, content creation, and editing in filmmaking.

- **Healthcare:** In the medical field, deep learning is used for tasks such as medical image analysis, early diagnosis of diseases, and personalized treatments. It helps detect conditions like cancer through image analysis, improves healthcare delivery, and assists in clinical research. Companies like Zebra Medical use deep learning for advanced medical diagnostics.

- **Fraud Detection:** In banking and finance, deep learning helps identify and prevent fraud by analyzing patterns in transactions and credit scores. Autoencoders and neural networks can detect anomalies in financial data, enhancing security and reducing financial losses for institutions.

- **Speech Recognition:** Deep learning has revolutionized speech-to-text and voice recognition technologies. It is used in smart assistants like Google Assistant and Amazon Alexa, providing accurate voice-to-text conversions and understanding spoken language in various contexts. Techniques like WaveNet have pushed the boundaries of text-to-speech synthesis.

- **Autonomous Vehicles:** Deep learning is crucial in developing autonomous driving systems. It enables vehicles to detect and interpret their surroundings, recognize objects like pedestrians and traffic signs, and make driving decisions, often outperforming human drivers in terms of safety and accuracy.

3.6 Process of Deep Learning

The process of deep learning involves several stages and various model categories, including supervised, unsupervised, and reinforcement learning. It typically involves the following

steps, as illustrated in Figure took.

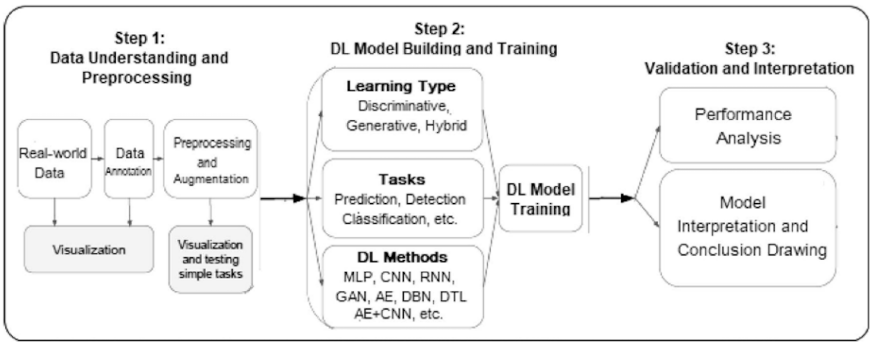


Figure 3.2: A typical DL workflow to solve real-world problems, which consists of three sequential stages (i) data understanding and preprocessing (ii) DL model building and training (iii) validation and interpretation from [20]

- **Data Collection:** Gather large datasets relevant to the task. Deep learning models require extensive data to learn effectively, whether it's images, text, audio, or other data types.
- **Data Preprocessing:** Clean and preprocess the data to make it suitable for training. This step includes tasks like normalizing data, handling missing values, and converting data into a format that can be fed into the model.
- **Model Selection:** Choose a neural network architecture that fits the problem. Common architectures include Convolutional Neural Networks (CNNs) for image recognition, Recurrent Neural Networks (RNNs) for sequential data like text and speech, and Transformer models for language processing.
- **Model Initialization:** Initialize the model's parameters (weights and biases) randomly or using predefined techniques. Proper initialization is crucial for the model's convergence during training.
- **Training:** Feed the training data into the model and adjust the parameters using optimization techniques like gradient descent. The model learns by minimizing the loss function, which measures the difference between the model's prediction and the actual labels. Training involves forward propagation (calculating the network's output) and backpropagation (adjusting weights based on the error).

- **Validation:** Use a validation dataset to monitor the model's performance during training. This helps in tuning hyperparameters and avoiding overfitting, ensuring that the model generalizes well to new, unseen data.
- **Testing:** Evaluate the model's performance on a separate test dataset to measure its accuracy and effectiveness. This step provides an unbiased evaluation of the final model.
- **Deployment:** After achieving satisfactory performance, deploy the trained model for practical use, such as in an application or service.
- **Inference:** The deployed model makes predictions on new data (inference), using the knowledge gained during training to produce output.
- **Fine-tuning and Optimization:** Continuously monitor the model's performance and fine-tune it by retraining with new data or using techniques like transfer learning to improve accuracy and efficiency.

3.7 Deep Learning Tools

Deep learning tools encompass a variety of frameworks, libraries, and platforms that facilitate the development, training, and deployment of deep learning models. Here are some popular tools:

- **Frameworks:** Popular frameworks include TensorFlow, Keras, and PyTorch, which are used for building and training deep learning models.
- **Libraries:** Essential packages such as NumPy, Pandas, and SciPy handle large amounts of data and are useful for working with multidimensional arrays, matrices, preprocessing, and feature engineering.
- **Data Visualization:** Popular libraries like Matplotlib, Seaborn, and TensorBoard support visualization, allowing users to create graphs, plots, and histograms to monitor and evaluate model performance.
- **Pre-trained Models and Model Zoos:** TensorFlow Hub and PyTorch Hub offer pre-trained models and machine learning modules to facilitate research reproducibility. Additionally, Hugging Face Transformers provides pre-trained models for natural language processing tasks such as text classification, translation, and summarization.

- **Cloud Platforms and Services:** Google Cloud AI Platform provides tools for building, deploying, and managing machine learning models at scale. Microsoft Azure Machine Learning also offers an end-to-end machine learning service for data scientists and developers to prepare data, build models, and deploy them at scale.
- **Automated Machine Learning (AutoML) Tools:** Google AutoML empowers developers without deep learning expertise to build machine learning models. Auto-Keras, an open-source AutoML library, makes machine learning more accessible to everyone.
- **Deployment and Production Tools:** ONNX (Open Neural Network Exchange) is an open format for representing deep learning models, enabling cross-framework compatibility. TensorFlow Serving is a high-performance system for serving machine learning models in production environments.
- **Specialized Tools for Research and Custom Architectures:** YOLO (You Only Look Once) is a real-time object detection system for recognizing various objects in images and videos. FastAI, a deep learning library built on top of PyTorch, provides high-level components that make it easy to achieve state-of-the-art results in standard deep learning applications.

3.8 Challenges and Limitations of Deep Learning

Deep learning has made significant strides in various domains, but it faces several challenges and limitations:

- **Data Requirements:** Deep learning models require vast amounts of labeled data for training. Collecting, labeling, and preprocessing this data can be costly and time-consuming. The performance of deep learning models heavily depends on data quality; noisy, biased, or incomplete datasets can lead to poor model performance.
- **Computational Resources:** Training deep learning models, especially deep neural networks, requires significant computational power, often involving GPUs or TPUs, which can be expensive and not accessible to everyone. Additionally, large models, such as GPT-3, consume substantial energy, raising concerns about their environmental impact.

- **Model Complexity:** Deep learning models can easily overfit, particularly when they are too complex relative to the training data, learning intricate patterns that do not generalize well to new data. These models have many hyperparameters that require careful tuning, making the process complex and requiring extensive experimentation.
- **Interpretability and Explainability:** Deep learning models, particularly deep neural networks, are often considered "black boxes" as they do not provide interpretable outputs. This lack of transparency makes it challenging to understand how the model makes decisions.
- **Generalization and Transfer Learning:** Deep learning models can struggle to generalize to new, unseen data, especially when there are significant differences between training and test data distributions. While transfer learning allows adaptation to new tasks with limited data, it has limitations; models trained on one domain may not always transfer well to drastically different domains.
- **Scalability and Deployment:** Scaling deep learning models for large-scale problems can be challenging in terms of computational resources and data management. Deploying deep learning models in real-time applications, such as autonomous driving, requires efficient models capable of making quick decisions, which is challenging for large, complex models.
- **Adversarial Attacks:** Deep learning models are vulnerable to adversarial attacks, where small, imperceptible perturbations to the input data can cause incorrect predictions. This is particularly concerning in safety-critical applications, such as security and autonomous vehicles.
- **Training Time and Resource Intensity:** Deep learning models can take days or even weeks to train, depending on model complexity and dataset size. The high-performance hardware requirements for training deep models limit accessibility and increase the barrier to entry for many organizations.

3.9 State-of-the-Art (SOTA) in Deep Learning

The state-of-the-art (SOTA) in deep learning is characterized by advancements across various domains such as natural language processing (NLP), computer vision, reinforcement learning, and generative models. Here's an overview of key areas and developments:

- **Computer Vision:** Transformer architectures have been extended to computer vision tasks, outperforming traditional convolutional neural networks (CNNs) in image classification, object detection, and segmentation. Generative Adversarial Networks

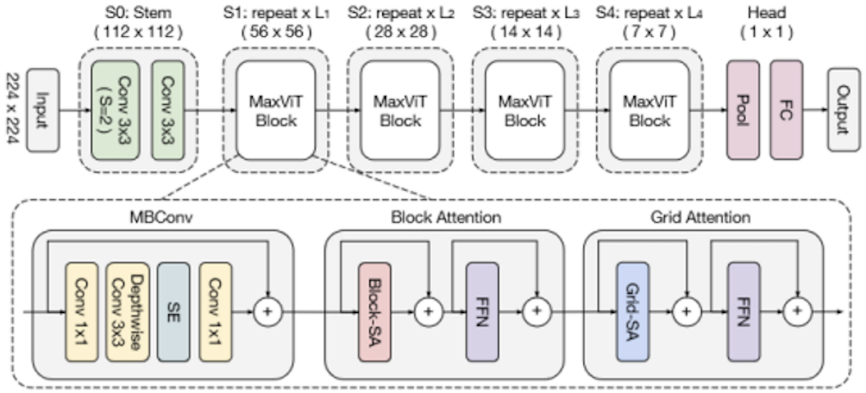


Figure 3.3: The architecture of MaxViT from [12]

(GANs) continue to lead in generating realistic images, with variants like StyleGAN3 producing high-quality, photorealistic images and advancing content generation capabilities shows the architecture of an image generator, DMF GAN.

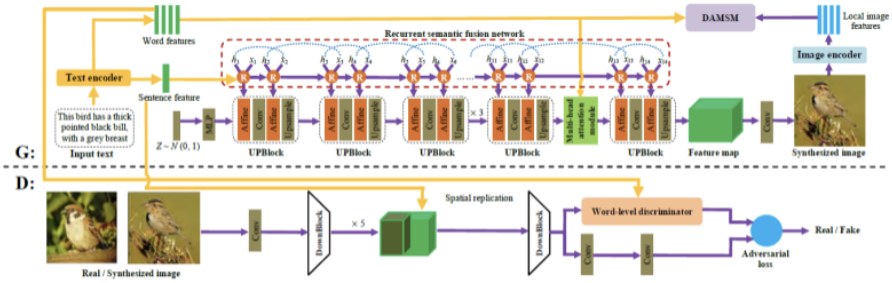


Figure 3.4: The architecture of DMF-GAN from [12]

- **Reinforcement Learning (RL):** Algorithms like Deep Q-Networks (DQN) and AlphaZero have achieved superhuman performance in games like Go, chess, and video games such as StarCraft II and Dota 2. RL is now also applied in complex robotic control tasks, enabling robots to learn from interactions with their environments to perform tasks like manipulation, locomotion, and autonomous navigation.

- **Speech Recognition:** In the realm of speech recognition, deep learning has surpassed traditional approaches by learning intricate features directly from raw au-

dio signals, bypassing the need for handcrafted features and complex preprocessing pipelines.

- **Self-Supervised and Unsupervised Learning:** Self-supervised learning methods have enabled the learning of meaningful representations from large amounts of unlabeled data, reducing dependence on labeled datasets. Unsupervised learning models like GPT-3 have shown that even without task-specific training, models can generalize well to various downstream tasks.

Summary of Deep Learning

Deep learning is a subset of machine learning that uses neural networks with multiple layers to automatically learn from vast amounts of data. Unlike traditional machine learning, it does not require manual feature engineering, allowing models to discover intricate patterns in data. This capability has led to breakthroughs across various domains, including natural language processing, computer vision, and speech recognition.

Key strengths of deep learning include handling unstructured data, scalability with large datasets, and applicability to complex tasks. However, it also faces challenges, such as high data and computational requirements, interpret ability issues, and susceptibility to adversarial attacks. Despite these challenges, deep learning continues to drive innovation in artificial intelligence, advancing capabilities in both research and real-world applications.

3.10 Computer Vision vs Deep Learning

Here is a table that highlights the key differences between computer vision and deep learning, emphasizing how deep learning automates and enhances the capabilities of computer vision task through neural networks.

Table 3.1: Comparative Analysis of Computer Vision and Deep Learning Approaches

Aspect	Computer Vision	Deep Learning
Definition	A field of artificial intelligence that focuses on enabling machines to interpret and make decisions based on visual data.	A subset of machine learning that uses neural networks to learn patterns from large datasets, often applied to various domains including vision.

Aspect	Computer Vision	Deep Learning
Approach	Traditional computer vision relies on manual feature extraction and rule-based algorithms to interpret visual data.	Uses neural networks, especially Convolutional Neural Networks (CNNs), to automatically learn features from raw data without manual intervention.
Data Requirements	Can work with smaller datasets as it often involves handcrafted features and rules.	Requires large amounts of labeled data for training to learn effectively, especially for complex tasks.
Feature Extraction	Involves manual feature extraction, where domain experts define key features like edges, corners, textures, etc.	Automates feature extraction through multiple layers of neural networks, learning high-level features directly from raw data.
Flexibility	Less flexible and often requires tuning specific to each task. Works well for simpler tasks like edge detection and object recognition with clear rules.	Highly flexible and capable of handling complex tasks like object detection, image segmentation, and facial recognition, adapting to various visual data forms.
Complexity	Simpler models that require less computational power, using algorithms like SIFT, SURF, and HOG for feature detection.	More complex models, like CNNs and GANs, require significant computational power and resources such as GPUs for training.
Performance	Effective for tasks where clear, predefined features can be used. Limited in performance when dealing with high-dimensional data or complex visual patterns.	Outperforms traditional computer vision methods in tasks like image classification, object detection, and facial recognition, especially in complex and high-dimensional scenarios.
Use Cases	Optical Character Recognition (OCR), barcode scanning, edge detection, object tracking.	Image classification, facial recognition, autonomous driving, medical image analysis, video analysis.
Interpretability	Generally more interpretable, as the algorithms use predefined rules, making it easier to understand the decision-making process.	Often seen as a 'black box,' making it challenging to interpret how decisions are made, especially in deep neural networks.
Learning Capability	Limited learning capability; improvements usually require manual tweaking of features and rules.	Capable of learning from data, adapting to different tasks, and improving performance through training on large datasets.

Summary

This comparative analysis shows that while Computer Vision and Deep Learning can both be applied to visual data interpretation, they serve different purposes based on the task's complexity, available resources, and interpretability requirements. Computer Vision meth-

ods are ideal for simpler, rule-based applications, whereas Deep Learning is suited for complex tasks that require scalability and the ability to handle vast amounts of data. Choosing between the two depends on the specific requirements of the application, with each approach having its unique strengths and limitations.

Chapter 4

Object Tracking and Re identification

4.1 Introduction

Object tracking and re-identification (Re-ID) are essential components in the field of computer vision, contributing significantly to a wide range of real-world applications, such as video surveillance, autonomous driving, and human-computer interaction. Object tracking aims to follow a specific object or multiple objects across video frames to understand their motion and behavior over time[5]. Re-identification, on the other hand, focuses on recognizing a particular object or person across different camera views or scenarios, ensuring consistent identity tracking even when the object exits and re-enters the field of view[11]. The primary challenge we aim to address is improving the efficiency of multi-camera tracking by integrating a re-identification model into the tracker. This addition will enable the system to handle complex scenarios where objects or individuals move across different camera views, maintaining accurate identity matching. In our upcoming work, we will delve into the theoretical background of object tracking and re-identification, review the methods used in these domains, and present our experimental results. Finally, we will compare our approach with the current state-of-the-art to evaluate its effectiveness and performance.

4.2 Theoretical Background

First, object tracking is the process of estimating the trajectory of an object in a video sequence while preserving its unique identity. The primary goal is to establish temporal

correspondences between detected objects in consecutive frames. The theoretical foundation of object tracking can be subdivided into two categories:

- **Detection-Based Tracking:** The foundation of modern tracking systems lies in robust object detection, where the object of interest is first detected in each frame using algorithms like convolutional neural networks (CNNs) or transformers[5]. Techniques used include:
  - **Hungarian Algorithm:** Uses data association to link detections to existing tracks[2].
  - **Kalman Filters:** A motion model commonly used to predict positions in subsequent frames[18].
- **Deep Learning Approaches:** Modern tracking algorithms leverage deep learning models, including recurrent neural networks (RNNs) for temporal sequence modeling and Siamese networks for similarity learning. These methods improve robustness to occlusions, appearance changes, and non-linear motion patterns[16].

Object Re-Identification (Re-ID) is the task of matching objects, primarily people, across non-overlapping cameras or different frames of a video. The key challenge is to identify and maintain object identities even when significant variations exist in appearance due to changes in viewpoint, lighting, or occlusion. For example, Person Re-Identification (Re-ID) [27] aims to match two pedestrian images from non-overlapping camera views. Due to large visual [12] variations in illumination, posture, viewpoint, misalignment, and background occlusions, person re-identification is a difficult task.

The theoretical background of Re-Identification (Re-ID) revolves around three main components:

- **Feature Representation:** Re-ID methods heavily depend on extracting discriminative features that are invariant to changes in appearance. Traditional approaches include Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG).
- **Distance Metrics and Similarity Functions:** Similarity between two images is typically measured using Euclidean or cosine distance. More sophisticated approaches include the Mahalanobis distance, which considers feature covariance, and learnable distance metrics using deep learning.



Figure 4.1: Examples of image pairs from the Market1501 dataset illustrating various challenges in person re-identification, including (a) variations in illumination, (b) differences in posture, (c) changes in viewpoint, (d) misalignment, and (e) background occlusion. Each pair contains images of the same person captured under these diverse conditions from [7].

- **Graph and Attention-Based Models:** Recent works incorporate graph convolutional networks (GCNs) and self-attention mechanisms to model interdependencies between multiple objects. These models improve robustness by considering spatial-temporal relationships and contextual information[13].

The combination of object tracking and re-identification is often referred to as multi-object tracking and re-identification (MOT-ReID) or multi-camera tracking (MCT). This integration relies on establishing correspondence between detected objects across multiple views or disjoint sequences. The core theoretical principles include:

- **Identity-Aware Tracking:** By incorporating re-identification features into the tracking pipeline, models become more robust to identity switches, occlusions, and camera hand-offs. This synergy between tracking and Re-ID allows for better handling of complex scenarios like crowded scenes or long-term tracking.

- **Joint Optimization Frameworks:** Joint frameworks optimize both tracking and re-identification objectives using end-to-end learning. For example, deep association networks (DAN) and deep affinity networks jointly learn tracking and Re-ID features, enabling robust identity preservation across challenging scenarios.

4.3 Methodology for Object Tracking and Re-Identification

4.3.1 Data Collection

For our data collection, we utilized videos captured from both surveillance cameras and cell phones, all recorded within the DSTD (Direction de la Stratégie et de la Transformation Digitale) office to simulate realistic usage scenarios. To accurately reflect real-world conditions, we used short video clips of up to 3 minutes each, focusing on challenging situations such as partial occlusions, where one individual temporarily blocks another. This setup allowed us to test the robustness and accuracy of our tracking system under complex conditions.

We categorized our test videos into four types based on specific scenarios:

- **Type 1 Videos:** In these clips, two people exit the camera's field of view and then re-enter, with a minor partial occlusion occurring when they overlap. This type of video tests whether the tracking algorithm can maintain a consistent ID for individuals when they leave and reappear.
- **Type 2 Videos:** These clips involve three individuals who frequently pass in front of and behind each other, creating numerous partial occlusions. Additionally, each person exits and re-enters the camera's view multiple times. This setup presents a significant challenge, as the frequent occlusions and re-entries can cause the tracker (e.g., BYTETrack) to assign new IDs due to each interruption.
- **Type 3 Videos:** Here, two individuals move from one camera's field of view to another. This scenario is difficult for traditional tracking algorithms because even slight differences in the bounding box or background can lead to the system misidentifying the same person as a new detection, thereby breaking the tracking continuity.
- **Type 4 Videos:** This is the most complex scenario, involving two non-overlapping camera fields. Initially, two people move between the camera views, and then a third person enters the scene unexpectedly. This test challenges the algorithm's ability to differentiate between previously tracked individuals and newly appearing ones, ensuring it can correctly maintain existing identities and assign a new ID to the newcomer.

These scenarios allow us to rigorously evaluate how well our deep learning-based tracking model handles complex real-world challenges, such as maintaining consistent IDs during occlusions, camera transitions, and the sudden introduction of new objects in the scene.

4.3.2 Object Detection-Based Tracking Implementation

The initial step we took was implementing an object detection-based tracking method using a tracking-by-detection algorithm. This approach involves detecting objects in each frame and then associating detections across consecutive frames to track their movement over time. To achieve this, we utilized an open-source library developed by Ultralytics, which provides several state-of-the-art object detection models, including the YOLO (You Only Look Once) series.

YOLO is a family of real-time object detection algorithms known for their speed and efficiency, making them highly suitable for computer vision applications such as autonomous driving, video surveillance, and robotics[4]. The core idea behind YOLO is to frame object detection as a single regression problem, directly predicting bounding boxes and class probabilities from entire images in a single forward pass. This differentiates it from traditional methods like R-CNN and its derivatives, which rely on multiple stages for region proposal and classification, resulting in higher computational cost.

For our project, we specifically used YOLOv9, an advanced version that addresses the issue of information loss in deep neural networks through novel techniques like Programmable Gradient Information (PGI) and Generalized Efficient Layer Aggregation Network (GELAN). These innovations allow YOLOv9 to retain crucial data across its depth, resulting in improved convergence and accuracy compared to its predecessors.

The following table took from [4] illustrates the performance of YOLOv9 on the COCO (Common Objects in Context) dataset, a standard benchmark for evaluating object detection models. It measures the model's accuracy using the mean Average Precision (mAP) metric across various object categories, providing a comprehensive assessment of its detection capabilities.

Object Category	**YOLOv9 mAP (%)**	**Previous YOLO Version mAP (%)**
Person	76.5	72.0
Vehicle	81.2	77.3
Animal	69.4	65.8
Appliance	63.7	60.2
Furniture	58.5	54.1

Table 4.1: Performance of YOLOv9 on the COCO Dataset: Mean Average Precision (mAP) Comparison

YOLOv9 uses a particular tracker algorithm, BYTETrack, which is a multi-object track-

ing algorithm known for its ability to handle complex tracking scenarios with high accuracy. BYTETrack works by associating detected bounding boxes across video frames and maintains identities for objects even when they are partially occluded or moving in complex patterns[26]. Below is a brief overview of BYTETrack's main features:

- **Tracking-by-Detection Approach:** BYTETrack operates by linking detection results across consecutive frames. It uses a combination of bounding box location and appearance information to match objects over time.
- **Handling Low Confidence Detections:** One of BYTETrack's key features is its ability to use low-confidence detections. While many traditional tracking algorithms discard low-confidence detections, BYTETrack keeps these detections to avoid losing track of objects temporarily due to slight confidence dips. This approach makes the tracker more robust and reduces the chances of missed detections.
- **Real-Time Performance:** BYTETrack is optimized for real-time applications, making it a good fit for the YOLO family of models, which are also designed for real-time object detection. When combined with YOLOv9, BYTETrack provides smooth and efficient object tracking, even in crowded and dynamic environments.

The figure below, illustrates a comparison between BYTETrack and other tracking algorithms. The evaluation was conducted on the BDD100K dataset, a diverse multi-category tracking benchmark designed specifically for autonomous driving scenarios. This dataset includes various challenging driving scenes, making it ideal for assessing the robustness and performance of tracking models under real-world conditions.

Tracker	split	mMOTA↑	mIDF1↑	MOTA↑	IDF1↑	FN↓	FP↓	IDs↓	MT↑	ML↓
Yu *et al.* [79]	val	25.9	44.5	56.9	66.8	122406	52372	8315	8396	3795
QDTrack [47]	val	36.6	50.8	63.5	**71.5**	108614	46621	**6262**	9481	3034
ByteTrack(Ours)	val	**45.5**	**54.8**	**69.1**	70.4	**92805**	**34998**	9140	**9626**	**3005**
Yu *et al.* [79]	test	26.3	44.7	58.3	68.2	213220	100230	14674	16299	6017
DeepBlueAI	test	31.6	38.7	56.9	56.0	292063	**35401**	25186	10296	12266
madamada	test	33.6	43.0	59.8	55.7	209339	76612	42901	16774	**5004**
QDTrack [47]	test	35.5	52.3	64.3	**72.3**	201041	80054	**10790**	17353	5167
ByteTrack(Ours)	test	**40.1**	**55.8**	**69.6**	71.3	**169073**	63869	15466	**18057**	5107

Figure 4.2: Performance Comparison of Object Tracking Algorithms Across Validation and Test Splits from [25].

The following picture is an example of BYTETrack combine with YOLOv9. In our work, the tracking component is implemented using a specific function provided by YOLOv9,

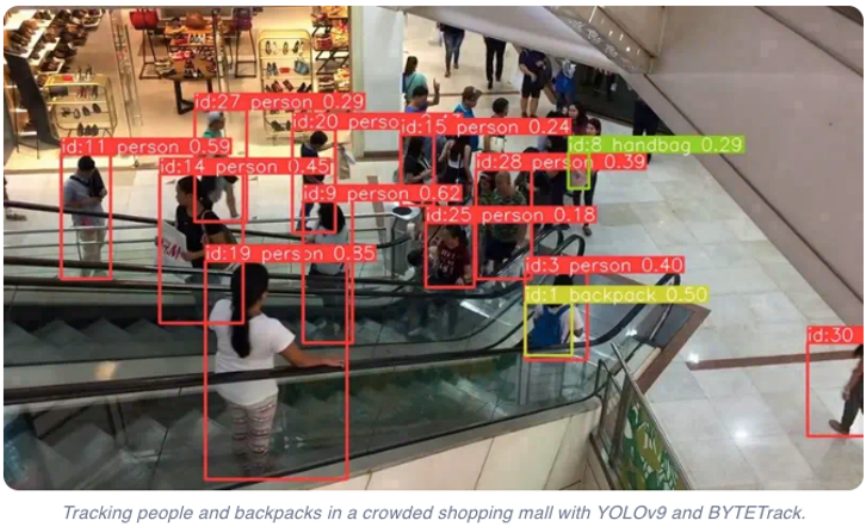


Figure 4.3: Real-Time Tracking of People and Objects in a Shopping Mall Using YOLOv9 and BYTETrack from [10].

namely yolo.track. This function automatically assigns unique IDs to each detected individual, allowing for efficient and seamless multi-object tracking across video frames.

4.3.3 TorchReID: Exploring the TorchReID Model Zoo

TorchReID is an open-source library specifically designed for deep learning-based person re-identification (Re-ID) using PyTorch [29]. It provides a comprehensive framework for developing, training, and evaluating person Re-ID models, making it a powerful tool for researchers and developers in the field of computer vision. TorchReID significantly contributed to our research, as it allowed us to evaluate various free models and select the most effective one for our specific needs. Below are some key features:

- **Model Zoo:** TorchReID includes a collection of pre-trained Re-ID models that can be easily used for research and deployment. This library supports a wide variety of convolutional neural network (CNN) architectures tailored for person Re-ID, including state-of-the-art models like ResNet, OSNet, and other advanced backbones.
- **Unified Data Management:** TorchReID has a robust data management module that supports multiple popular Re-ID datasets such as Market1501, DukeMTMC-reID, and CUHK03. It allows seamless integration of these datasets, and users can create new ones by defining simple configurations. The library supports both image-based and video-based Re-ID, making it suitable for a variety of applications.

- **Visualization Tools:** The library includes tools for visualizing ranking results and activation maps, which help in understanding how well a model has learned to distinguish between different identities. This is crucial for analyzing performance and making further improvements to the models.

TorchReID is primarily used for person re-identification in surveillance, robotics, and retail analytics, where maintaining consistent identities across multiple camera views is essential. Researchers can leverage it to benchmark new algorithms or fine-tune existing models on custom datasets.

A feature extractor is a component of a machine learning model, typically used in the early layers of a deep neural network, that transforms raw input data (e.g., images or audio) into a compact, high-level representation of relevant features. It captures essential patterns such as edges, textures, shapes, or other distinguishing characteristics that are useful for further processing or classification tasks. With TorchReID, we leveraged the model zoo and used pre-trained models as feature extractors. Since we didn't have the time or resources to build a large and robust dataset for fine-tuning, we opted to use existing models to extract features for our downstream tasks.

The following image illustrates some of the models available in the TorchReID model zoo. We evaluated almost all the available models, but many did not meet our specific require-

MSMT17 (`combineall=True`) -> Market1501 & DukeMTMC-reID

Model	# Param (10^6)	GFLOPs	Loss	Input	Transforms
resnet50	23.5	2.7	softmax	(256, 128)	`random_flip` , `color_ji`
osnet_x1_0	2.2	0.98	softmax	(256, 128)	`random_flip` , `color_ji`
osnet_x0_75	1.3	0.57	softmax	(256, 128)	`random_flip` , `color_ji`
osnet_x0_5	0.6	0.27	softmax	(256, 128)	`random_flip` , `color_ji`
osnet_x0_25	0.2	0.08	softmax	(256, 128)	`random_flip` , `color_ji`
osnet_ibn_x1_0	2.2	0.98	softmax	(256, 128)	`random_flip` , `color_ji`
osnet_ain_x1_0	2.2	0.98	softmax	(256, 128)	`random_flip` , `color_ji`

Figure 4.4: Model Comparison for Person Re-Identification on MSMT17, Market1501, and DukeMTMC-reID Datasets from [29]

ments. However, one model that consistently delivered strong performance was ResNet50, which stood out for its accuracy and robustness. We extracted features from individual frames of the videos and stored them in CSV files, allowing us to easily manipulate and analyze the data for various downstream tasks.

4.3.4 Dimensionality Reduction with Principal Component Analysis (PCA)

Our dataset initially consisted of 2,048 feature columns extracted using ResNet50, along with a large volume of raw data. To enhance performance, we applied Principal Component Analysis (PCA), a widely used dimensionality reduction technique in statistics and machine learning. PCA simplifies high-dimensional datasets by retaining the most significant features, thereby reducing computational complexity and making the data easier to visualize and interpret.

This step was crucial in our work, as it allowed us to reduce the feature space to a much lower dimension, enabling us to visualize the quality of the extracted features. Through PCA, we could identify effective feature extractors and distinguish them from less effective ones, a critical step since using a poor feature extractor limits progress, regardless of the algorithms applied afterward.

4.3.5 K-Nearest Neighbor (KNN) for Classification

K-Nearest Neighbor (KNN) is a simple, yet powerful supervised machine learning algorithm used for classification and regression tasks. The main idea behind KNN is to classify a data point based on how its neighbors are classified, making it a type of instance-based learning.

In our work, we utilized KNN for classification using CSV files containing the extracted features along with their corresponding IDs. Since KNN is a supervised learning algorithm, we partially trained it on every 50 frames collected by our program. Once trained, it classifies each feature based on the IDs it learned. This method enables us to perform re-identification effectively, offering a novel approach to maintaining consistent identities across frames.

KNN has several advantages, including:

- **Simplicity:** KNN is easy to understand and implement, making it accessible for a wide range of applications.
- **Versatility:** It can be used for both classification and regression problems, enhancing its utility in various tasks.

4.3.6 Anomaly Detection with Autoencoders

When analyzing Type 4 videos, we encountered a significant challenge, particularly in detecting new individuals. Traditional classification methods, such as K-Nearest Neighbors

(KNN) or other conventional classifiers, proved inadequate for reliably distinguishing between individuals. To overcome this limitation, we developed and implemented an anomaly detection approach leveraging an autoencoder.

An autoencoder is a type of artificial neural network used in unsupervised learning to learn efficient representations of input data.[3] It typically performs dimensionality reduction, feature extraction, and data reconstruction by compressing the input data into a lower-dimensional latent space and then reconstructing it back to its original form. In the context of anomaly detection, an autoencoder learns to reconstruct the normal patterns in data during training. When it encounters anomalous data (data significantly different from the normal patterns), the reconstruction error (difference between input and output) is typically much higher. This reconstruction error is used as a measure to identify anomalies.

The following illustration depicts the architecture of an autoencoder.

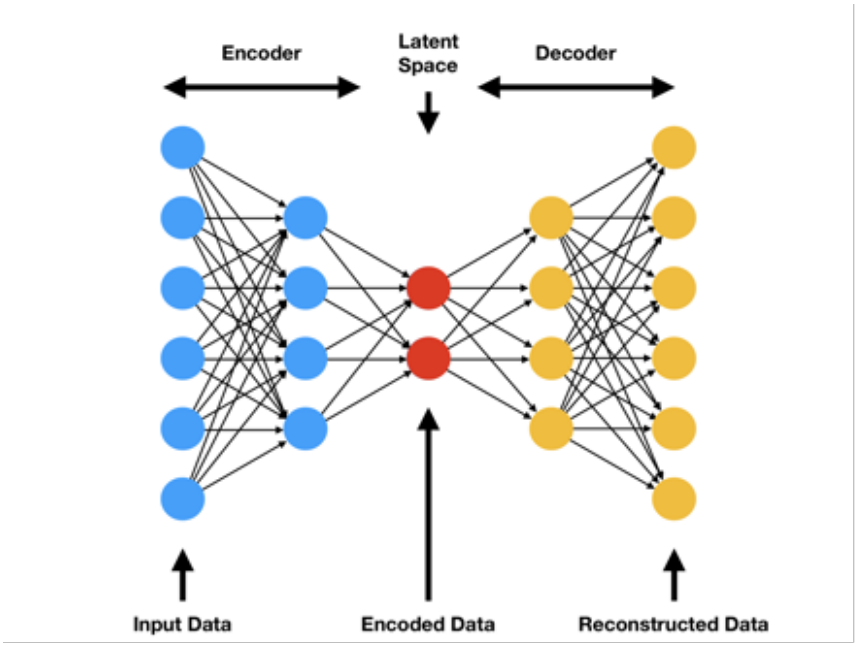


Figure 4.5: Architecture of an Autoencoder: Encoding, Latent Space, and Decoding Process from [22]

An autoencoder is composed of three main parts:

- **Encoder:** The encoder is responsible for compressing the input into a lower-dimensional latent space representation. It maps the input data X to a hidden representation h (also called the bottleneck or latent space) using one or more layers.

- **Latent Space (Bottleneck):** This is the compressed representation of the input data. The latent space captures the most important features, removing redundant information. The bottleneck forces the network to learn a simplified, abstract representation of the input.
- **Decoder:** The decoder takes the compressed latent representation and attempts to reconstruct the original input as closely as possible. It performs the reverse operation of the encoder, mapping the hidden representation h back to a reconstruction.
- **Reconstruction Loss:** The quality of the reconstruction is measured using a loss function, such as mean squared error (MSE) or cross-entropy, depending on the type of input data. The goal of the autoencoder is to minimize this loss.

In our work, we use the autoencoder as an anomaly detector. First, it is trained on the saved features of known individuals. When a new detection occurs, the autoencoder encodes and decodes the features. If the features belong to a previously unseen individual, the reconstruction loss will increase significantly, indicating that the features do not match the learned patterns. This approach enabled us to enhance the performance of our re-identification model, making it effective even in challenging scenarios such as various types of occlusions.

4.4 Result and Discussion

4.4.1 Feature Extraction for Re-Identification

Feature extraction is a key step in the process of re-identifying individuals. The aim is to transform raw images of individuals into discriminating feature vectors, enabling identities to be distinguished effectively. For this purpose, we used a convolutional neural network model, ResNet50, pre-trained on specific re-identification datasets (Market1501 and DukeMTMC-reID).

A feature vector from a ResNet-50 model represents a compressed, high-dimensional summary of an input image. This vector is typically obtained from one of the last layers of the network, often before the final classification layer. It encapsulates important information about the image, capturing various features such as textures, shapes, and patterns. The features are hierarchical, meaning that earlier layers capture simple patterns (e.g., edges, colors), while deeper layers capture more complex structures (e.g., objects, scenes).

To better understand the quality and relevance of the extracted features, we projected them into a lower-dimensional space using Principal Component Analysis (PCA). Additionally, we used a graph to optimize the selection of the most effective dimensionality reduction technique, enabling us to preserve a significant amount of variance in the data.

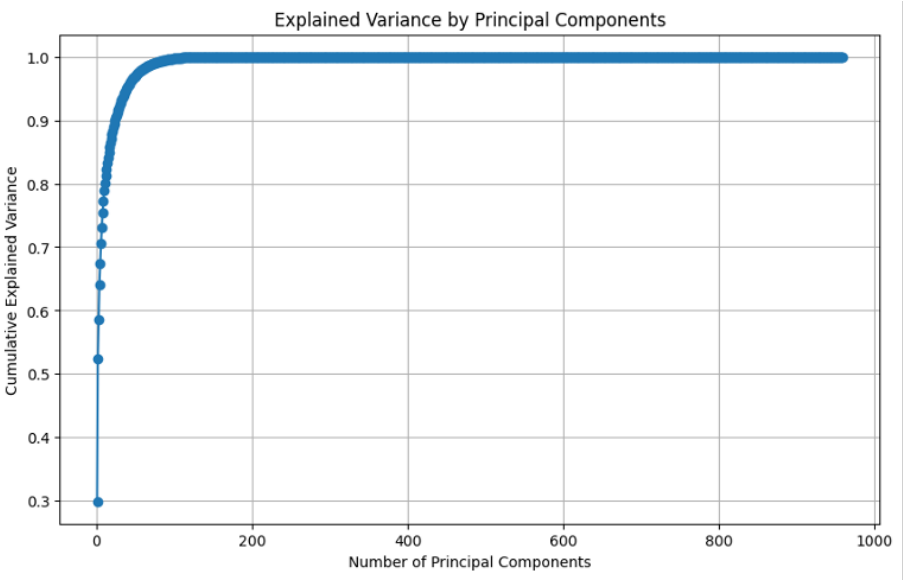


Figure 4.6: Cumulative Explained Variance by Number of Principal Components

We used ResNet50 to extract features and plotted the cumulative explained variance of Principal Component Analysis (PCA). It appears that most of the variance (over 90) can be explained by a relatively small number of principal components (around 50-100). This indicates that dimensionality reduction is feasible without significant information loss.

The "elbow" in the plot serves as a useful point to choose an optimal number of components. Beyond this point, the gain in explained variance is minimal, suggesting that additional components may not be necessary for the analysis. Based on this observation, we chose to retain only 50 components in PCA.

These dimensionality reduction techniques allow us to visualize the distribution of individuals in feature space and evaluate whether identical individuals are well grouped and separated from others. We applied our feature extractor to a Type 1 video, and the resulting plot is shown below.

In this PCA visualization, we observe how the features extracted using ResNet-50 are separated based on the different IDs corresponding to individuals in the images. The scatter plot shows data points projected onto the first two principal components, with each color representing a different person (ID). The distribution indicates that ResNet-50

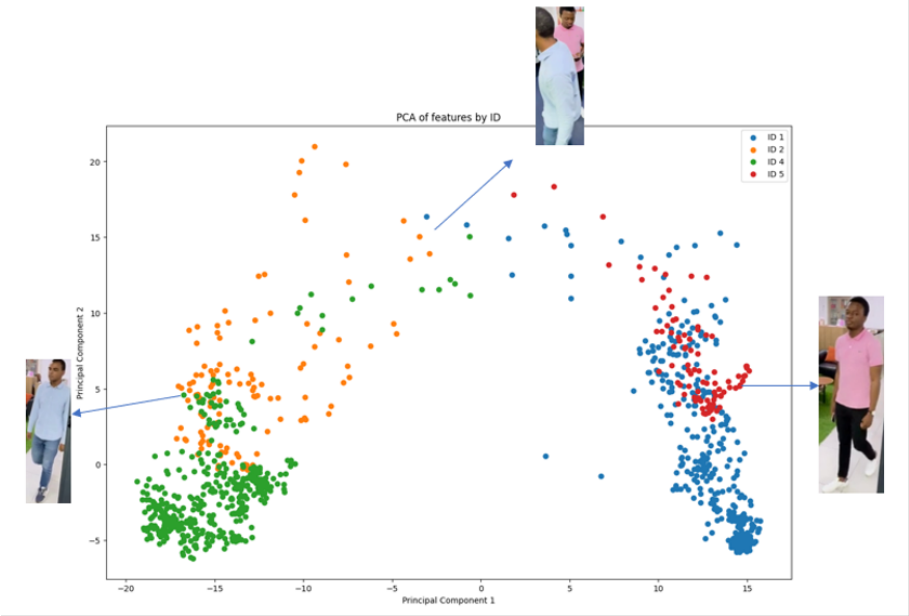


Figure 4.7: PCA Visualization of Feature Clusters by Person ID with Sample Images

effectively captures distinctive features from the images, resulting in clusters corresponding to different individuals.

- **Cluster Separation:** The distinct clusters formed by the points demonstrate that ResNet-50 can differentiate between various individuals based on their extracted features. Each group represents a unique ID, showing that the model can distinguish between individuals even in a lower-dimensional space after dimensionality reduction.
- **Overlap Between Clusters:** Some overlap or proximity between clusters, where features from different IDs appear near each other, may indicate shared visual characteristics. This overlap can result from occlusions, similar poses, or background similarities in the original images, making some features less distinctive.
- **Outliers:** A few scattered points deviating from the main clusters may represent variations in the images due to factors such as different camera angles, lighting conditions, or partial occlusions (e.g., an individual holding an object or having part of their body out of the frame).

For comparison, the following plot Figure 4.7 illustrates a poor feature extractor, where clusters are not clearly separated, indicating that distinctive features are not adequately captured.

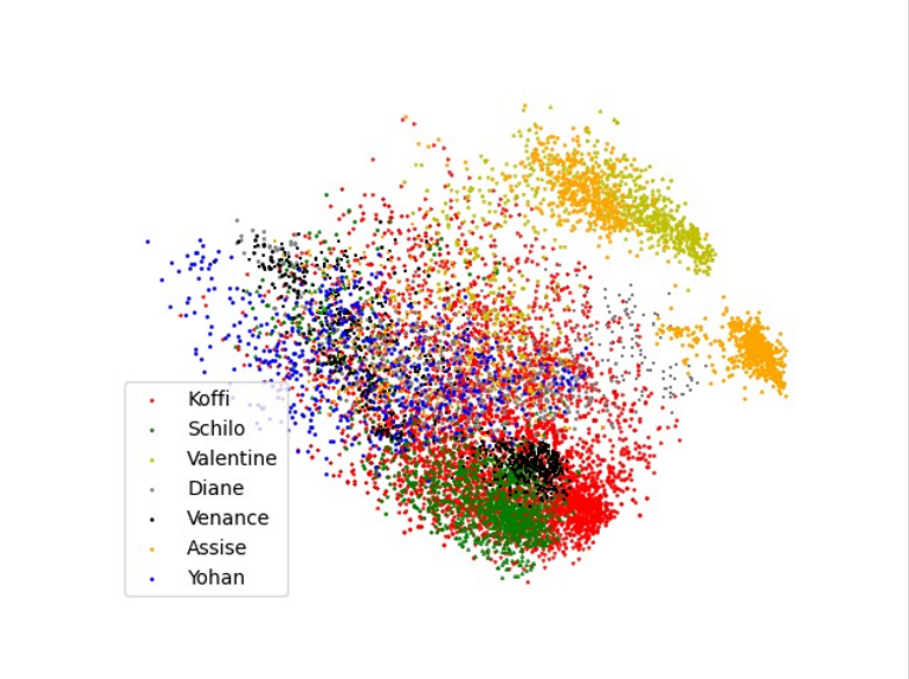


Figure 4.8: Feature Distribution of Individuals Visualized by Color-Coded Clusters

In this Figure 4.7, the data points represent features extracted for different individuals, with each color corresponding to a specific person. Unlike the previous plot, this one show significant overlap between the clusters, indicating that the feature extractor struggled to distinguish between individuals effectively.

4.4.2 Challenges in Identifying New Individuals

Determining whether an individual is a new person presents the most challenging aspect of our work. To address this, we experimented with various algorithms. We utilized Type 4 video data, training several classifiers on the first two individuals who entered the camera's field of view. Subsequently, we assessed the classifiers' ability to assign an ID to new individuals and analyzed the prediction probabilities associated with these classifications. However, these algorithms did not perform adequately.

However, the results revealed that all the machine learning models—ranging from simpler classifiers like K-Nearest Neighbors (KNN), Logistic Regression, and Support Vector Machines (SVM) to more complex ensembles such as Gradient Boosting, AdaBoost, and Random Forests—failed to identify the new person as "unrecognized" or novel. Instead, these algorithms confidently misclassified the new individual as one of the already known

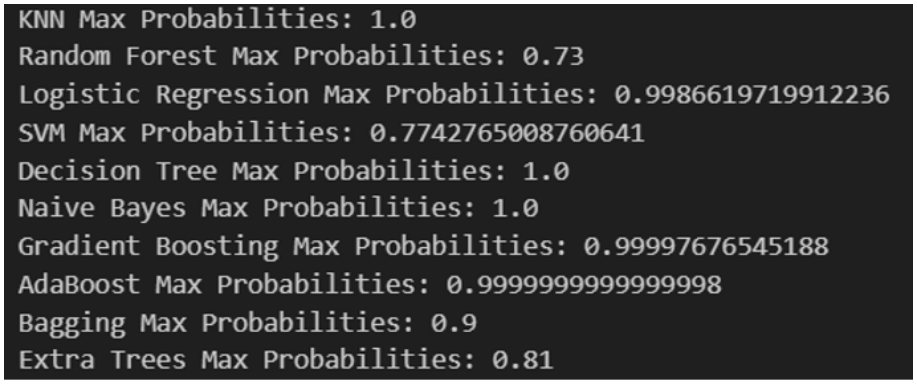


Figure 4.9: Maximum Predicted Probabilities by Various Machine Learning Models

identities with high probabilities, as shown in the image.

For instance:

- **KNN, Decision Tree, and Naive Bayes:** These models yielded maximum probabilities of 1.0, suggesting total certainty in assigning an incorrect existing ID to the new person.
- **Random Forest and Extra Trees:** These algorithms were slightly more conservative, with probabilities of 0.73 and 0.81, respectively, but still failed to identify the individual as new.
- **Advanced Models (AdaBoost and Gradient Boosting):** Even more advanced models, such as AdaBoost and Gradient Boosting, predicted with near-perfect certainty (e.g., 0.9999999) that the new person was someone already known.

These results demonstrate a common limitation of traditional machine learning classifiers when used in open-set recognition tasks. Such algorithms are typically optimized for closed-set problems, where all possible identities are known during training. As a result, they are ill-equipped to handle situations where novel instances (in this case, new individuals) appear during testing. The algorithms are biased towards predicting one of the known classes, which explains why they assigned high probabilities to incorrect matches rather than flagging the new individual as unfamiliar.

4.4.3 Anomaly Detection Approach for Identifying New Individuals

To identify the presence of a new individual, we developed a novel approach inspired by our extensive research efforts. This approach leverages anomaly detection using an autoencoder, which has demonstrated promising results in our experiments.

Specifically, we trained the autoencoder on known features, allowing it to learn the patterns associated with familiar individuals. When presented with new features that the autoencoder has not encountered before, the resulting reconstruction error is significantly higher. This increased reconstruction error serves as an indicator, allowing us to determine whether the individual is new or previously known.

In this example involving Type 4 video data, we trained an autoencoder on the features extracted from the first two individuals who appeared in the first camera's field of view. Subsequently, a third person appeared in the second field of view along with the original two individuals. The goal of this experiment was to evaluate the autoencoder's ability to distinguish between well-reconstructed features (which correspond to known individuals) and poorly reconstructed features (which would indicate new, unseen individuals or anomalies).

The contingency table provides a clear view of the autoencoder's reconstruction accuracy across different test cases.

People	**1st known person**	**2nd known person**	**3rd unknown person**	**Total**
Good reconstruction	179	174	119	**472**
Bad reconstruction	1	2	128	**131**
Total	**180**	**176**	**247**	**603**

Table 4.2: Contingency Table of Reconstruction Quality Across Known and Unknown Individuals

First Individual: The autoencoder was trained on the first individual's features, and the confusion matrix shows that out of 180 instances, 179 were reconstructed correctly. This indicates that the autoencoder has learned the distribution of this person's features very well. Only 1 bad reconstruction was observed, reflecting a minor error but still an overall strong performance.

Second Individual: Similarly, for the second individual, the autoencoder correctly reconstructed 174 instances with only 2 bad reconstructions. This slight increase in error compared to the first person could be due to natural variation in the features of the second individual or a slight difference in how the autoencoder learned the data. Nevertheless, the majority of reconstructions were accurate, indicating the autoencoder's strong generalization for this person.

Unseen Individual (Anomaly Detection): When presented with features from a new, unseen individual (i.e., a person the autoencoder has never encountered during train-

ing), the confusion matrix highlights a significant increase in bad reconstructions. Out of the total 247 instances, only 119 were correctly reconstructed, while 128 were poorly reconstructed. This is a strong indicator that the autoencoder is recognizing this person as an anomaly. Since the autoencoder was not trained on this individual's features, it struggles to reconstruct the input accurately, leading to a high number of errors.

Conclusion: The confusion matrix provides compelling evidence that the autoencoder can effectively detect anomalies. The sharp contrast between high reconstruction accuracy for known individuals and significant errors for new individuals shows that the model can differentiate between familiar and unfamiliar patterns. Moving forward, this approach could be further refined by tuning the autoencoder's architecture or incorporating additional anomaly detection techniques to enhance its performance.

4.4.4 Person Re-Identification Using K-Nearest Neighbors (KNN)

To achieve accurate person re-identification, we employed a classifier that predicts known features and assigns them to their corresponding IDs. Specifically, we used the K-Nearest Neighbors (KNN) algorithm for this task. KNN was chosen due to its simplicity and effectiveness in handling feature-based classification. By comparing the feature vectors of new inputs to those of known individuals, KNN was able to successfully assign the correct identities.

The classifier performed well, achieving an accuracy of nearly 90, demonstrating its capability in distinguishing between different individuals based on their feature representations. This high accuracy indicates that KNN is well-suited for re-identifying individuals whose feature patterns are similar to those observed during training, making it a valuable component of our re-identification system.

The following figure provides a visual representation of the KNN algorithm in our re-identification system.

4.4.5 Discussion of Results and Limitations

The results obtained show that the developed algorithm performs better, which serves as an initial validation of the objectives set. Re-identification performance can be observed through the contingency table. However, although the results are encouraging, the program's overall performance remains limited.

The main difficulty lies in the sensitivity of the threshold used for re-identification. When this threshold is set too low, the algorithm tends to consider new individuals as already

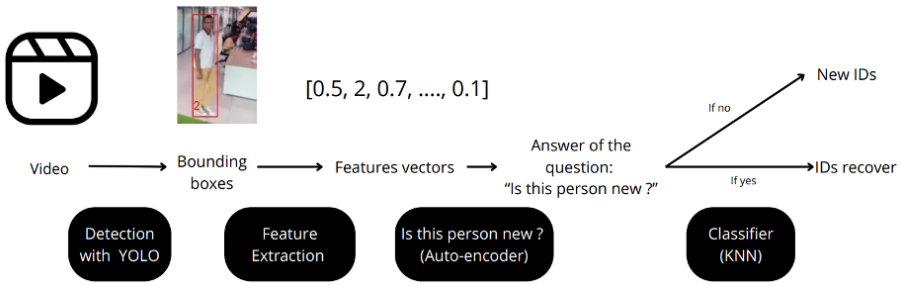


Figure 4.10: Performance Comparison of Object Tracking Algorithms Across Validation and Test Splits.

seen, generating re-identification errors. Conversely, if the threshold is set too high, the system fails to recognize individuals it has already encountered, reducing its accuracy. These limitations prevent the algorithm from being fully usable in real-life applications due to excessive errors.

However, once the algorithm correctly identifies a person as "already seen," the classification operates reliably. The individual classification is performed by a KNN or neural network, which accurately assigns an identity to the tracked person.

Although the project is far from complete, it has laid the foundation for a functional algorithm. Our approach was to simplify the problem initially by limiting the number of individuals to be re-identified, maintaining consistent lighting conditions, and using a fixed camera. This allowed us to progressively test the algorithm in more challenging environments, where it demonstrated a certain robustness, even under less controlled conditions.

Summary

In Chapter 4, we developed and evaluated an algorithm for person re-identification and ID tracking in video feeds. We implemented various machine learning techniques, including K-Nearest Neighbors (KNN) and autoencoders, to address the challenges associated with re-identifying individuals across frames and distinguishing between known and unknown individuals.

The chapter details the step-by-step approach taken, from feature extraction using ResNet50 to anomaly detection with an autoencoder for identifying new individuals. The results indicate that while the algorithm shows promising initial accuracy and reliability, particularly in controlled environments, several limitations prevent it from being fully applicable in real-life scenarios. Key challenges include the sensitivity of the re-identification threshold and the need to optimize the algorithm for dynamic and uncontrolled conditions.

Despite these challenges, the algorithm demonstrates robust classification performance when correctly identifying known individuals. This foundational work establishes a basis for further improvements and adjustments, with the potential to refine the algorithm for broader real-world applications.

Conclusion

During this project, our main objective was to develop a solution for re-identifying people captured in videos from scratch [19]. One of the first important steps was breaking down the problem into two distinct phases: detecting individuals, followed by classification. Subsequently, this breakdown evolved to include a mechanism for identifying new individuals and then classifying previously seen individuals. This original approach was our own initiative, as we did not have any pre-existing examples to rely on. We also specified and narrowed down the problem to focus on short-term re-identification, meaning individuals seen during the same day. If a person returned with significantly different physical characteristics, such as a new hairstyle or radically different clothing, the algorithm would consider them as new individuals. This decision stemmed from a limitation we identified in our working environment: the installed IP camera did not allow for sufficient facial distinction to ensure reliable long-term re-identification. Thus, the distinction between short-term and long-term re-identification, and the choice of the short-term, was entirely our initiative. As the project progressed, we found that the task was more complex than anticipated. Effectively re-identifying people in videos is a problem that requires significantly more material and time resources than we had available. However, despite these challenges, we were able to develop several algorithms to address the problem and conducted tests by recording and analyzing videos. Our work therefore consisted not only of coding algorithms and testing them through various trials, but also, and more importantly, deciding how to approach this complex problem in a field where we had no concrete examples and where much of the work had to be invented.

Bibliography

[1] Luca Bertinetto, Jack Valmadre, João F. Henriques, Andrea Vedaldi, and Philip H. S. Torr. Fully-convolutional siamese networks for object tracking, 2021. URL `https://arxiv.org/abs/1606.09549`. 5

[2] Minh-Quan Dao and Vincent Frémont. A two-stage data association approach for 3d multi-object tracking, 2021. URL `https://arxiv.org/abs/2101.08684`. 30

[3] Cole Stryker Dave Bergmann. What is an autoencoder? *IBM*, 2023. 38

[4] Glenn Jocher, Matthew Noyce, Muhammad Rizwan, Francesco Mattioli, and Burhan. Yolov9 benchmarks. *Ultralytics*, Year. 33

[5] Nico Klingler. Why computer vision is difficult. *Viso AI*, Year. 7, 9, 29, 30

[6] J.R. Larus. Evolution of computing. In H. Werthner et al., editors, *Introduction to Digital Humanism*. Springer, Cham, 2024. doi: 10.1007/978-3-031-45304-5_3. 7

[7] Zhouchen Lin, Liang Wang, Jian Yang, Guangming Shi, Tieniu Tan, Nanning Zheng, Xilin Chen, and Yanning Zhang. Pattern recognition and computer vision. Year. 31

[8] B. D. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the 7th International Joint Conference on Artificial Intelligence*, pages 674–679. IJCAI, 1981. 5

[9] Nehemiah Musa, Abdulsalam Gital, Nahla Aljojo, Haruna Chiroma, S. Adewole, Hammed Mojeed, Nasir Faruk, A. Abdulkarim, Yusuf Folawiyo, James Ogunmodede, Abdulkarim Oloyede, Olawoyin Abiodun, Ismaeel Sikiru, and Ibrahim Katb. A systematic review and meta-data analysis on the applications of deep learning in electrocardiogram. *Journal of Ambient Intelligence and Humanized Computing*, 14, 2022. doi: 10.1007/s12652-022-03868-z. 17

[10] Sushma Nagdeote, Simrandeep Singh, Aditya Kalambe, Aaron Gomes, and Amardeep Bhupatwar. Proceedings of the 12th international conference on soft computing for problem solving. Year. 35

[11] Soon Xiao Xi Natasha and Sutthiphong Srigrarom. Person re-identification for multi-camera, multi-object tracking on robotic platforms. In *2023 15th International Conference on Information Technology and Electrical Engineering (ICITEE)*, pages 121–126, 2023. doi: 10.1109/ICITEE59582.2023.10317712. 29

[12] M.H.M. Noor and A.O. Ige. A survey on deep learning and state-of-the-art applications. Year. 25, 30

[13] Ioannis Papakis, Abhijit Sarkar, and Anuj Karpatne. Gcnnmatch: Graph convolutional neural networks for multi-object tracking via sinkhorn normalization, 2021. URL `https://arxiv.org/abs/2010.00067`. 31

[14] N.G. Polson and V.O. Sokolov. Deep learning. Year. 17, 18

[15] Amr Ramadan, Ahmed Algarhy, Hossam Zidan, and Nihal Darraj. Advanced detection of sucker rod pump faults using computer vision and dynamometer card analysis. 2024. doi: 10.21203/rs.3.rs-4517115/v1. 14

[16] Kanchana Ranasinghe, Sahan Liyanaarachchi, Harsha Ranasinghe, and Mayuka Jayawardhana. Extending multi-object tracking systems to better exploit appearance and 3d information, 2019. URL `https://arxiv.org/abs/1912.11651`. 30

[17] Kevin J. Ryan. The tech that powers your self-driving car might be built using people playing games on their phones. Year. 9

[18] Rahmad Sadli. Object tracking: 2-d object tracking using kalman filter in python. *Object Detection*, 2020. 30

[19] I.H. Sarker. Deep learning: A comprehensive overview on techniques, taxonomy, applications, and research directions. Year. 48

[20] Iqbal Sarker. Deep learning: A comprehensive overview on techniques, taxonomy, applications and research directions. *SN Computer Science*, 2, 2021. doi: 10.1007/s42979-021-00815-1. 21

[21] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, January 2015. ISSN 0893-6080. doi: 10.1016/j.neunet.2014.09.003. URL http://dx.doi.org/10.1016/j.neunet.2014.09.003. 18

[22] Hongteng Wang, Xuewei Liu, Liyong Ma, and Yong Zhang. Anomaly detection for hydropower turbine unit based on variational modal decomposition and deep autoencoder. Year. 38

[23] Y. Ye, Z. Wang, J. Hao, Y. Hou, S. Xiao, W. Zeng, and Y. Luo. Deepfusion feature representation learning with hard mining center-triplet loss for person re-identification. *Journal Name*, Year. 14

[24] Yilin Ye, Jianing Hao, Yihan Hou, Zhan Wang, Shishi Xiao, Yuyu Luo, and Wei Zeng. Generative ai for visualization: State of the art and future directions. Year. 15

[25] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. Research on bytedance and huazhong university of science and technology. Year. 34

[26] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box, 2022. URL https://arxiv.org/abs/2110.06864. 34

[27] C. Zhao, X. Lv, Z. Zhang, W. Zuo, J. Wu, and D. Miao. Deep fusion feature representation learning with hard mining center-triplet loss for person re-identification. Year. 30

[28] Liang Zheng, Liyue Shen, Lu Tian, Shengjin Wang, Jingdong Wang, and Qi Tian. Scalable person re-identification: A benchmark. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, December 2015. 5

[29] Kaiyang Zhou. Torchreid model zoo. Year. 35, 36

[30] Kaiyang Zhou and Tao Xiang. Torchreid: A library for deep learning person re-identification in pytorch, 2019. URL https://arxiv.org/abs/1910.10093. 6